

PROGRAMMING IN HASKELL

1



Topic 14.1 - Invariants From Concept to Haskell

What they are, why they matter, and how Haskell encodes them

What is an Invariant?

2c

A condition that always holds

An invariant is a property of a system that remains true before and after every operation.

It is a guarantee — not a comment, not a convention, and not an assumption.

When an invariant holds we say the system is in a valid state.

Bank balance

Never drops below its minimum — guaranteed after every transaction.

Calendar date

Day stays 1–31, month 1–12. No valid operation produces an out-of-range date.

Stack size

The element count equals the number of pushes minus pops — always.

Invariants in Software

3

Every major abstraction relies on invariants to remain coherent.

Data Structures

A binary search tree assumes $\text{left} < \text{root} < \text{right}$ at every node. Violate it once and search is silently broken.

State Machines

A TCP connection cannot be CLOSED and ESTABLISHED simultaneously. The state type makes the invalid state inexpressible.

Algorithms

Merge sort maintains a sorted invariant on every sub-list. The merge step is only correct because both halves are already sorted.

Why Invariants Matter

4

Correctness

Code that maintains its invariants cannot produce a whole class of bugs. Violations are caught at the boundary, not buried deep in a call stack.

Refactoring Safety

When invariants are explicit and enforced, you can safely change internals. Callers cannot accidentally break what they cannot construct.

Living Documentation

A type that enforces its invariant is documentation that never goes stale. The constraint is read, checked, and enforced by the compiler.

Types as Invariant Enforcers

5

Haskell's type system makes some invariants impossible to violate — not because you tested them, but because ill-formed programs don't compile.

What you get for free

- No null — use Maybe to make absence explicit
- No mixing of incompatible units (metres vs feet)
- Exhaustive pattern matching — all cases handled
- Distinct types for logically distinct concepts

Example

```
-- Maybe makes absence explicit
find :: [Int] -> Int -> Maybe Int

-- Can't pass a raw String as an Email
newtype Email = Email String

-- Compiler catches missing cases
describe :: Bool -> String
describe True  = "yes"
describe False = "no"
```

Smart Constructors

6

A smart constructor hides the raw data constructor and validates the invariant at the point of creation — so an invalid value can never be built.

The pattern

1. Define the type inside a dedicated module
2. Export only the smart constructor — not the data constructor
3. Return Maybe or Either to signal invalid input

```
module Calendar (Day, mkDay, getDay) where

-- Constructor hidden from callers
data Day = Day Int

mkDay :: Int -> Maybe Day
mkDay n
  | n >= 1 && n <= 31 = Just (Day n)
  | otherwise         = Nothing

getDay :: Day -> Int
getDay (Day n) = n
```

newtype and Phantom Types

7

Two lightweight tools for encoding invariants with zero runtime cost.

newtype

Wraps a type to give it a distinct identity at compile time. An Email and a plain String are different types — you cannot pass one where the other is expected.

```
newtype Email = Email String

sendTo :: Email -> IO ()
-- Cannot pass a raw String
```

Phantom Types

A type parameter that carries compile-time information but holds no value at runtime. Useful for tagging validated vs unvalidated data without overhead.

```
data Validated
data Raw

newtype Tagged t a = Tagged a

validate :: Tagged Raw String
         -> Maybe (Tagged Validated String)
```


Typeclass Laws as Invariants

8

Typeclass instances are contracts. The laws are the invariants every conforming instance must preserve — forever.

Functor

```
fmap id    ≡ id
fmap (f . g) ≡ fmap f .
fmap g
```

Mapping preserves structure. The container is never altered by fmap.

Monoid

```
mempty <> x  ≡ x
x <> mempty  ≡ x
(x <> y) <> z ≡ x <> (y <>
z)
```

Combination is associative and mempty is a genuine neutral element.

Monad

```
return a >>= f  ≡ f a
m >>= return    ≡ m
(m >>= f) >>= g
  ≡ m >>= (f >=> g)
```

return is a neutral wrap. Sequencing is associative.

Where Types Aren't Enough

9

The type system is powerful but bounded. Some invariants live in logic, not in shape.

Types CAN enforce

- A value is never null (Maybe)
- Two arguments share the same type
- A list is non-empty (NonEmpty)
- A number is always positive (smart constructor)

Types CANNOT enforce

- A sort function actually produces a sorted list
- Encode and decode are true inverses
- A number stays in range after arbitrary arithmetic
- Two data structures stay mutually consistent

For these, we need property-based testing - covered in next slide deck

Recap: What We Covered

10

1. Invariants are guarantees — conditions that must always hold
2. Haskell's type system enforces many invariants at compile time
3. Smart constructors and newtypes push invariants to the boundary
4. Typeclass laws are formal, checkable invariants
5. Some invariants live in logic — and need explicit verification

